Task Submission Report: Deep Learning Image Classifier

Author: Darapaneni Chaturya
Task: Transfer Learning Image Classification (CIFAR-10 & ResNet-18)
Framework: PyTorch

1. Overview & Objective

The objective of this task is to build and evaluate a deep learning image classifier using transfer learning in PyTorch. The CIFAR-10 dataset was chosen, consisting of 60,000 32x32 color images across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

2. Architecture & Transfer Learning

- **Pretrained Backbone:** ResNet-18 pretrained on ImageNet (using `ResNet18_Weights.DEFAULT`).
- **Feature Extraction:** All feature extractor layers were frozen (`requires_grad = False`) to preserve pre-trained feature representation.
- **Classifier Head:** Replaced the final fully connected layer (`model.fc`) with a linear layer mapping from 512 input features to 10 output classes.
- **Data Augmentation & Preprocessing:**
    - Resized input images to 224x224 to match standard ResNet input resolution.
    - Random Horizontal Flips and Random Rotations (±10°) applied to training data.
    - Normalized pixel values using ImageNet mean `[0.485, 0.456, 0.406]` and std `[0.229, 0.224, 0.225]`.

3. Training & Inference Code

3.1 Training Script (train.py)

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torchvision.models import resnet18, ResNet18_Weights
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform_train = transforms.Compose([
    transforms.Resize(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

transform_test = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
download=True, transform=transform_train)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=32,
shuffle=True, num_workers=2)

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)

for param in model.parameters():
    param.requires_grad = False

num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, 10)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)

num_epochs = 3
train_loss_history, train_acc_history = [], []
```

```
for epoch in range(num_epochs):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    epoch_loss = running_loss / len(trainloader)
    epoch_acc = 100.0 * correct / total
    train_loss_history.append(epoch_loss)
    train_acc_history.append(epoch_acc)
    print(f"Epoch [{epoch+1}/{num_epochs}] | Loss: {epoch_loss:.4f} |
Accuracy: {epoch_acc:.2f}%")

torch.save(model.state_dict(), 'cifar_resnet18.pth')
```

3.2 Inference Script (inference.py)

```
import torch
from torchvision import transforms
from torchvision.models import resnet18
from PIL import Image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
'ship', 'truck']

model = resnet18(weights=None)
model.fc = torch.nn.Linear(model.fc.in_features, 10)
model.load_state_dict(torch.load('cifar_resnet18.pth', map_location=device))
model = model.to(device)
model.eval()

transform = transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

```
def predict_image(image_path):
    image = Image.open(image_path).convert('RGB')
    input_tensor = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        outputs = model(input_tensor)
        probabilities = torch.nn.functional.softmax(outputs, dim=1)
        confidence, predicted_idx = torch.max(probabilities, 1)
    print(f"Predicted Class: {classes[predicted_idx.item()]}
({confidence.item()*100:.2f}%)")
```

4. Repository & Setup (README.md)

```
# CIFAR-10 Transfer Learning with ResNet-18

## Environment Setup
```bash
git clone https://github.com/your-username/cifar10-resnet18.git
cd cifar10-resnet18
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts ctivate
pip install torch torchvision matplotlib pillow
```

Running Instructions
- Train model: `python train.py`
- Run inference: `python inference.py --image path_to_image.jpg`
```